



NetVision

Plateforme décisionnelle de supervision réseau et de
gestion des incidents — Network Operations Center,
Huawei Maroc

DOCUMENTATION & PRÉPARATION À LA SOUTENANCE

PFE 2026 · **10** secteurs · **120** équipements · **30** techniciens · **300** incidents

Sommaire

1 Résumé du projet

2 Architecture générale

3 Modèle de données (schéma en étoile)

- Dimensions

- Faits

- Vues

4 Fonctionnalités détaillées

- 4.1 Authentification simplifiée (login.html)

- 4.2 Dashboard analytique (noc_admin.html → page Dashboard)

- 4.3 Carte interactive géolocalisée (noc_admin.html → page Carte)

- 4.4 Dispatch assisté (GET /api/dispatch/, POST /api/dispatch/assigner)

- 4.5 Planning Gantt journalier (noc_admin.html → page Gantt, GET /api/gantt)

- 4.6 Interface technicien terrain (tech.html)

- 4.7 Clôture automatique & redispach des incidents bloqués

- 4.8 Traçabilité complète (audit trail)

5 Questions de soutenance possibles

- 5.1 Questions générales / contexte du projet

- 5.2 Architecture & choix techniques

- 5.3 Base de données & modélisation

- 5.4 Algorithme de dispatch

- 5.5 Fonctionnalités temps réel (clôture auto, pause, Gantt)

- 5.6 KPI et calculs

- 5.7 Authentification & sécurité

- 5.8 Limites, robustesse et perspectives d'évolution

1 Résumé du projet

NetVision est une plateforme décisionnelle (BI) de supervision réseau et de gestion des incidents, développée pour le Network Operations Center (NOC) de Huawei Maroc dans le cadre d'un PFE 2026. Elle couvre tout le cycle de vie d'un incident réseau : détection (incident simulé dans `fact_incidents`), dispatch assisté vers un technicien, suivi terrain en temps réel via une interface mobile-friendly, clôture (manuelle ou automatique), et traçabilité complète de chaque action dans un audit trail.

Le projet couvre 10 secteurs géographiques du Maroc, 120 équipements Huawei, 300 incidents simulés et 30 techniciens terrain. Il repose sur un entrepôt de données PostgreSQL en schéma en étoile, une API Flask, et deux interfaces web (NOC Admin et Technicien Terrain), accessibles depuis une page de connexion simplifiée.

L'objectif pédagogique principal est de démontrer une chaîne complète **génération de données → ETL → entrepôt de données → API → restitution décisionnelle (dashboard, carte, Gantt)**, avec en plus une couche opérationnelle temps réel (dispatch, suivi terrain, clôture automatique) qui dépasse le cadre d'un simple reporting BI.

2 Architecture générale

```
generate_data.py → data/ (CSV, JSON, schema_dwh.sql)
    |
    ▼
etl_pipeline.py → extract → transform → load → PostgreSQL (bi_telecom)
    |
    ▼
app.py (Flask, port 5000) → ~30 endpoints REST (JSON)
    |
    ├── login.html      (choix de profil Admin / Technicien)
    ├── noc_admin.html  (Dashboard, Carte, Incidents, Techniciens, Gantt)
    └── tech.html       (interface technicien terrain)
```

Couches techniques : Python 3.11 / Flask / Flask-CORS / psycopg2 côté backend ; PostgreSQL 15 côté stockage (schéma en étoile) ; HTML5 / CSS3 / JavaScript vanilla côté frontend (aucun framework, pour rester simple à déboguer et à présenter) ; Leaflet.js pour la carte ; Chart.js pour les graphiques.

Pourquoi pas de framework frontend (React/Vue) ? Le projet privilégie la simplicité de déploiement (un seul fichier HTML autoportant par interface, ouvrable directement dans un navigateur sans étape de build) et la lisibilité pour la maintenance : tout le code d'une page est visible dans un seul fichier.

3 Modèle de données (schéma en étoile)

Dimensions

- **dim_secteur** : les 10 secteurs géographiques (nom, région, latitude/ longitude du centre).
- **dim_equipement** : 120 équipements Huawei (code, nom, type — BTS, routeur, antenne, etc. —, modèle, statut ACTIF/MAINTENANCE/HORS_SERVICE, secteur, coordonnées GPS).
- **dim_technicien** : 30 techniciens (matricule, nom/prénom, compétences — tableau JSON —, secteur, coordonnées GPS, statut DISPONIBLE / EN_INTERVENTION / EN_PAUSE / INDISPONIBLE, note de performance `note_perf`, horaires 09h-18h, `ticket_actif_id` qui pointe vers le ticket en cours s'il y en a un).
- **dim_temps** : calendrier (jour, semaine, mois, trimestre, jour de la semaine, week-end) — utilisé pour les agrégations temporelles.

Faits

- **fact_incidents** : un incident = une panne détectée sur un équipement. Colonnes clés : `code_incident` (format INC-YYYYMM-XXXX), `type_incident` (PANNE_COMPLETE, COUPURE_FIBRE, DEGRADATION, etc.), `severite` (CRITIQUE/MAJEUR/MINEUR), `statut` (OUVERT → EN_COURS → RESOLU/INCOMPLET), `duree_estimee_min` (tirée aléatoirement dans une plage dépendant du type au moment de la génération), `duree_resolution_min` (rempli uniquement à la clôture), géolocalisation, et `technicien_assigne_id/nom` (rempli au dispatch, vidé à la résolution).
- **fact_tickets** : un ticket = une intervention créée lors du dispatch d'un incident. Contient toute la timeline (`date_assignment`, `date_arrivee_site`, `date_debut_intervention`, `date_fin_intervention`), le `statut_terrain` (ASSIGNE → EN_ROUTE → SUR_SITE → EN_COURS → [EN_PAUSE] → COMPLETE/INCOMPLET), `duree_intervention_min` (calculée à la clôture), `note_intervention`, `cause_incomplete`, et les deux colonnes liées à la pause terrain : `pause_debut` (timestamp de début de pause) et `duree_pause_min` (cumul des minutes de pause, ajouté à `duree_estimee_min` à la reprise).
- **ticket_historique** : une ligne par transition de statut d'un ticket (`statut_avant`, `statut_apres`, `updated_by` = ADMIN/TECHNICIEN/SYSTEME, `cause`, `commentaire`, `updated_at`) — c'est l'audit trail.

Vues

- `v_kpi_global`, `v_incidents_par_secteur`, `v_performance_techniciens`, `v_gantt_aujourd'hui` : vues SQL utilitaires pour simplifier certaines agrégations (même si plusieurs endpoints recalculent directement via requêtes SQL pour avoir un contrôle fin sur les filtres temps réel).

Pourquoi un schéma en étoile et pas un schéma normalisé classique (3NF) ? Le schéma en étoile sépare clairement les dimensions (référentiels qui changent peu : secteurs, équipements, techniciens) des faits (événements : incidents, tickets), ce qui simplifie et accélère les requêtes analytiques (agrégations, jointures simples) — c'est l'approche standard en data warehousing / BI.

4 Fonctionnalités détaillées

4.1 Authentification simplifiée (login.html)

Objectif : donner accès aux deux interfaces sans construire un système de comptes individuels pour 30 techniciens, hors périmètre validé du PFE (prévu pour une phase ultérieure par une équipe dev dédiée).

Fonctionnement : `login.html` affiche deux boutons profil — "NOC Admin" et "Technicien Terrain". Au clic, un objet `{role: 'ADMIN'|'TECHNICIEN'}` est stocké dans `sessionStorage` sous la clé `nv_user`, puis redirection vers `noc_admin.html` ou `tech.html`. Chacune de ces deux pages vérifie au chargement (`window.addEventListener('load', ...)`) la présence de `nv_user` dans `sessionStorage` ; si absent, redirection forcée vers `login.html`. Le bouton "Déconnexion" (`confirmLogout()`) demande une confirmation puis supprime `nv_user` et redirige vers `login.html`.

Côté `tech.html`, après être entré dans l'interface, un sélecteur ("Identification → Sélectionner votre profil") permet de choisir lequel des 30 techniciens on incarne — les techniciens sont regroupés par secteur (`<optgroup>`) et triés alphabétiquement pour faciliter la recherche.

4.2 Dashboard analytique (noc_admin.html → page Dashboard)

Objectif : donner une vue d'ensemble en temps réel de l'état du réseau et des opérations.

Fonctionnement : au chargement de la page (et toutes les 15 secondes via `setInterval(refreshKPI, 15000)`), le frontend appelle `GET /api/kpi`. Cet endpoint exécute trois requêtes d'agrégation SQL : 1. sur `fact_incidents` : total, incidents ouverts, en cours, critiques/ majeurs/mineurs non résolus, taux de résolution (`RESOLU+FERME / total`), MTTR moyen (`AVG(duree_resolution_min)`), en minutes et en heures, durée estimée moyenne par intervention (`AVG(duree_estimee_min)`) ; 2. sur `fact_tickets` : nombre de tickets actifs/complets/incomplets, note d'intervention moyenne, distance moyenne parcourue ; 3. sur `dim_technicien` : nombre de techniciens disponibles / en intervention / indisponibles.

Les 7 indicateurs clés (incidents critiques, MTTR réel, taux de résolution, techniciens disponibles, durée estimée moyenne, etc.) sont affichés dans des `.kpi-card`. En dessous, 8 graphiques Chart.js sont alimentés par des endpoints dédiés : `/api/incidents/par-secteur`, `/api/incidents/par-reseau`, `/api/incidents/par-heure` (distribution horaire), `/api/incidents/evolution` (évolution sur 7 jours), `/api/performance/techniciens` (comparaison des techniciens), et la répartition par sévérité calculée côté frontend à partir des compteurs KPI.

4.3 Carte interactive géolocalisée (noc_admin.html → page Carte)

Objectif : visualiser spatialement équipements, incidents actifs et techniciens sur le territoire marocain, et lancer le dispatch directement depuis la carte.

Fonctionnement : la carte utilise Leaflet.js 1.9.4 avec un fond de tuiles sombre CartoCDN (`dark_all`), centré sur le Maroc. Au premier accès à la page (`initMap()`), déclenché paresseusement par `goPage('carte', ...)` pour ne pas charger Leaflet inutilement), un appel à `GET /api/carte` renvoie en un seul JSON : la liste des secteurs, des équipements (avec statut et coordonnées), des incidents actifs (avec sévérité) et des techniciens (avec statut et position). Chaque catégorie est affichée avec une icône Leaflet `divIcon` colorée selon son statut/sévérité (rouge = CRITIQUE, ambre = MAJEUR, etc.). Un panneau de filtrage permet d'afficher/masquer chaque couche. Cliquer sur un marqueur d'incident ouvre le panneau de dispatch (`showIncDP`) décrit ci-dessous ; cliquer sur un équipement affiche ses informations (`showEqDP`).

4.4 Dispatch assisté (GET /api/dispatch/<id>, POST /api/dispatch/assigner)

Objectif : aider le superviseur NOC à choisir rapidement le technicien le plus pertinent pour un incident donné.

Fonctionnement (très important pour la soutenance — détailler précisément) :

- Quand l'admin clique sur un incident `OUVERT` (sur la carte ou dans la liste Incidents), le frontend appelle `GET /api/dispatch/<incident_id>`.
- Cet endpoint sélectionne tous les techniciens avec `statut='DISPONIBLE' AND ticket_actif_id IS NULL`, calcule pour chacun la **distance à vol d'oiseau** en kilomètres avec la formule $111.0 * \sqrt{((lat_tech - lat_incident)^2 + (lng_tech - lng_incident)^2)}$ (1° de latitude/longitude ≈ 111 km — approximation suffisante à l'échelle du Maroc, pas de calcul de distance routière réelle), et un booléen `meme_secteur` (le technicien appartient-il au même secteur que l'incident ?).
- Le tri se fait avec `ORDER BY secteur_id = secteur_incident DESC, distance_km ASC` : **critère 1 = même secteur, critère 2 = distance la plus courte**. La réponse renvoie séparément `meme_secteur` (tous les techniciens du même secteur, triés par distance) et `autres_secteurs` (les 5 plus proches hors secteur).
- Le frontend affiche une liste de candidats classés (max 4), avec pour chacun : distance, **note de performance** (`note_perf`, *, basée sur l'historique de notation des interventions précédentes), badge "même secteur" et compétences. Le premier de la liste est mis en évidence (`.tdc.top`) comme **suggestion**.
- L'admin **clique** sur le candidat retenu (souvent la suggestion, mais il peut en choisir un autre — l'algorithme est une **aide à la décision**, pas une automatisation totale) puis clique sur "Créer & Assigner Ticket", ce qui appelle `POST /api/dispatch/assigner`.
- Cet endpoint recrée la distance, calcule `duree_estimee_min` de l'incident, et vérifie les horaires : si l'heure actuelle est avant 9h ou après 18h, ou si `heure_actuelle + duree_estimee > 18h`, l'assignation n'est **pas insérée** : l'API renvoie `confirmation_request: true` avec l'heure de fin estimée, et le frontend affiche une popup demandant confirmation pour reporter l'intervention au **lendemain 09h00** (`POST /api/dispatch/assigner-j1`).
- Si l'horaire est compatible, le ticket est créé dans `fact_tickets` (`statut_terrain='ASSIGNE'`, `date_assignment=now`), l'incident passe en `EN_COURS` avec `technicien_assigne_id/nom` renseignés, et le technicien passe en `statut='EN_INTERVENTION'`, `ticket_actif_id=<id du ticket>` (il devient indisponible pour un nouveau dispatch).

À noter : `note_perf` est affichée comme aide à la décision mais **n'intervient pas dans le ORDER BY** — seuls le secteur et la distance trient automatiquement la liste. C'est un point que le jury peut creuser ("votre algorithme est-il vraiment basé sur 3 critères pondérés ?") : la réponse honnête est que le tri automatique repose sur 2 critères (secteur, distance), et la note de performance est une **information affichée** pour guider le choix humain final.

4.5 Planning Gantt journalier (noc_admin.html → page Gantt, GET /api/gantt)

Objectif : visualiser sur une frise horaire (09h-18h) les interventions de tous les techniciens pour un jour donné, détecter les retards et exporter le planning.

Fonctionnement : `GET /api/gantt?date=YYYY-MM-DD` renvoie, pour chaque technicien, ses informations (nom, secteur, horaires) et le ticket actif ce jour-là (statut terrain, sévérité, timeline complète, `pause_debut`, `duree_pause_min`, `raison_indisponibilite`). Le frontend dessine pour chaque technicien une barre positionnée selon `date_assignment` et dimensionnée selon la durée logique du type de panne ; la couleur dépend du `statut_terrain` (bleu = EN_ROUTE, ambre = SUR_SITE, cyan = EN_COURS, vert = COMPLETE, rouge = INCOMPLET). Le sélecteur de date (`<input type="date">` , boutons précédent/suivant) permet de naviguer dans l'historique. Un bouton "Export" génère un export Excel/CSV du planning affiché. Toutes les 30 secondes (page Gantt active), `runFermerRetards()` appelle `POST /api/tickets/fermer-retards` (voir 4.7) et rafraîchit l'affichage, donnant l'impression d'un planning "vivant".

4.6Interface technicien terrain (tech.html)

Objectif : permettre au technicien de suivre et mettre à jour son intervention en cours depuis un mobile.

Fonctionnement : après sélection de son profil, le technicien voit son ticket actif (s'il en a un) avec les informations de l'incident (équipement, type de panne, sévérité, durée estimée) et une carte Leaflet centrée sur le lieu d'intervention. La mise à jour du statut se fait en **4 étapes séquentielles**, chacune via `PATCH /api/tickets/<id>/statut` avec `statut_terrain = EN_ROUTE → SUR_SITE → EN_COURS → COMPLETE / INCOMPLET` : - **EN_ROUTE** : le technicien a accepté/commence le déplacement (statut initial après dispatch, ou premier clic). - **SUR_SITE** : enregistre `date_arrivee_site = now`. - **EN_COURS** : enregistre `date_debut_intervention = now` (si pas déjà renseigné). - **COMPLETE** ou **INCOMPLET** : enregistre `date_fin_intervention = now` et calcule `duree_intervention_min = temps écoulé depuis date_assignment` (vision "tout inclus" : trajet + intervention). Si **COMPLETE**, une note d'intervention (1-5) peut être saisie, l'incident passe en **RESOLU** avec `duree_resolution_min` renseigné, et le technicien redevient **DISPONIBLE**. Si **INCOMPLET**, une `cause_incomplete` est **obligatoire** (l'API renvoie une erreur sinon), l'incident passe en **INCOMPLET** (visible sur la carte admin pour redispatch — voir 4.7).

Pause terrain avec décompte visuel (`POST /api/tickets/<id>/pause`, `action=START|END`) : à tout moment pendant **EN_COURS**, le technicien peut déclencher **une seule pause par ticket** (15-30 min typiquement). **START** passe `statut_terrain='EN_PAUSE'`, enregistre `pause_debut=now`, et passe le technicien en `statut='EN_PAUSE'` dans `dim_technicien`. L'interface affiche un décompte visuel. **END** calcule la durée réelle écoulée depuis `pause_debut`, l'ajoute à la fois à `duree_pause_min` (cumul affiché) et à `duree_estimee_min` du ticket (la pause **prolonge** l'estimation, pour que le calcul de retard — voir 4.7 — ne considère pas la pause comme un retard), remet `statut_terrain='EN_COURS'` et le technicien en **EN_INTERVENTION**. Une tentative de reprendre une pause déjà utilisée sur le même ticket est rejetée (HTTP 400).

Le technicien reçoit aussi des notifications sonores/visuelles (rafraîchissement toutes les 10 secondes côté `tech.html`) si un nouveau ticket lui est assigné (y compris une popup "Reprise demain à 09h00" si le ticket a été planifié en J+1).

4.7Clôture automatique & redispatch des incidents bloqués

Objectif : libérer automatiquement un technicien si son intervention dépasse la durée estimée, sans attendre une action manuelle, et permettre de redispatcher les incidents non résolus.

Fonctionnement : `POST /api/tickets/fermer-retards` est appelé toutes les 30 secondes par `noc_admin.html` (boucle `setInterval`, page Dashboard et page Gantt). Il sélectionne tous les tickets dont `statut_terrain` n'est ni **COMPLETE**, ni **INCOMPLET**, ni **EN_PAUSE** (un ticket en pause n'est jamais considéré en retard, puisque la pause a déjà prolongé `duree_estimee_min`), puis calcule en Python `date_assignment + duree_estimee_min` et compare à l'heure actuelle. Pour chaque ticket en retard : - le ticket passe `statut_terrain='INCOMPLET'`, `statut='INCOMPLET'`, avec `cause_incomplete='Durée estimée dépassée – clôture automatique NOC'`, `duree_intervention_min` calculée, et `updated_by='SYSTEME'` ; - le technicien repasse `statut='DISPONIBLE'`, `ticket_actif_id=NULL` (libéré pour un nouveau dispatch) ; - une ligne `ticket_historique` est ajoutée avec `updated_by='SYSTEME'`.

L'**incident** associé reste visible sur la carte avec un statut bloquant. Le superviseur NOC peut alors cliquer sur "Débloquer" (`POST /api/incidents/<id>/debloquer`), qui vérifie que l'incident est bien **INCOMPLET**, puis le remet **OUVERT** (et vide `ticket_id`, `technicien_assigne_id/nom`) — l'incident redevient candidat à un nouveau dispatch via le flux normal (4.4). C'est le mécanisme de **"redispatch conditionnel soumis à validation du superviseur"** : le système ne redispatche jamais seul, il remet juste l'incident dans la file.

Par ailleurs, `GET /api/incidents/escalade` liste les incidents CRITIQUE/MAJEUR encore **OUVERT** et non assignés depuis plus de 30 minutes — affiché et rafraîchi toutes les 2 minutes (`setInterval(buildEscalade, 120000)`) pour attirer l'attention du superviseur sur les incidents "oubliés".

4.8Traçabilité complète (audit trail)

Objectif : garder un historique exhaustif de chaque ticket pour analyse a posteriori (et pour la timeline interactive).

Fonctionnement : la table `ticket_historique` reçoit une ligne à **chaque** transition de statut, quelle que soit son origine : - `updated_by='TECHNICIEN'` pour les transitions via `tech.html` (PATCH `/statut`, et START/END de pause) ; - `updated_by='ADMIN'` si un admin force une transition (ex. `POST /api/tickets/<id>/forcer-cloture`) ; - `updated_by='SYSTEME'` pour la clôture automatique (4.7).

Chaque ligne contient `statut_avant`, `statut_apres`, `cause` (si incomplet), `commentaire` (texte libre, ex. "Fin pause — 18 min ajoutées à la durée estimée") et `updated_at`. `GET /api/tickets/<id>/historique` renvoie cette liste pour un ticket donné, affichée dans `noc_admin.html` sous forme de timeline interactive (icônes/couleurs par type de transition).

5 Questions de soutenance possibles

Les questions sont regroupées par thème. Pour chacune, une piste de réponse basée sur l'implémentation réelle est proposée — à reformuler avec vos propres mots le jour J.

5.1 Questions générales / contexte du projet

Q — Pourquoi ce sujet, et quel problème concret résout-il ?

R — Le NOC gère manuellement (ou avec des outils dispersés) la détection d'incidents, l'affectation des techniciens et le suivi des interventions. NetVision centralise ces trois étapes dans un seul outil avec une vue temps réel, ce qui réduit le temps de réaction (dispatch assisté plutôt que recherche manuelle du technicien le plus proche) et donne une traçabilité complète pour le reporting.

Q — Quel est le périmètre exact de votre PFE — qu'avez-vous développé vous-même vs ce qui est simulé/hors périmètre ?

R — Tout le code (génération de données, ETL, API Flask, les trois interfaces) est développé pour ce PFE. Les **données** (incidents, positions GPS, historiques) sont **simulées** par `generate_data.py` car l'accès aux systèmes de supervision réels (type U2000) n'était pas possible. L'**authentification individuelle** par technicien est volontairement hors périmètre (cf. 5.6).

Q — Combien de temps a duré le développement, et comment avez-vous organisé le travail ?

R — Réponse personnelle — décrire vos itérations (ex. d'abord le modèle de données et l'ETL, puis l'API, puis le dashboard, puis le dispatch et le suivi terrain, puis les ajustements UI/UX).

5.2 Architecture & choix techniques

Q — Pourquoi PostgreSQL et pas MySQL ou MongoDB ?

R — PostgreSQL est un SGBD relationnel robuste, gratuit, avec un excellent support des agrégations SQL complexes (`FILTER` , fenêtres) nécessaires aux KPI, et `psycopg2` s'intègre simplement à Flask. Un modèle relationnel en étoile convient bien à des données structurées (incidents, tickets, techniciens) avec des relations claires — un NoSQL type MongoDB n'apporterait pas d'avantage ici et compliquerait les agrégations analytiques.

Q — Pourquoi Flask et pas Django ou FastAPI ?

R — Flask est minimaliste et suffisant pour une API REST d'une trentaine d'endpoints sans besoin d'ORM complexe ni d'admin auto-généré (Django), et il est plus familier/rapide à mettre en place pour un projet de cette taille qu'une stack FastAPI + ORM async. `Flask-CORS` gère simplement les appels cross-origin depuis les fichiers HTML statiques.

Q — Pourquoi du JavaScript vanilla et pas React/Vue ?

R — Chaque interface est un seul fichier HTML autoportant (HTML+CSS+JS), ouvrable directement dans un navigateur sans étape de build/npm — plus simple à déployer, à déboguer et à présenter en soutenance (tout le code d'une page tient dans un fichier). Pour la taille du projet (3 pages), React aurait ajouté de la complexité sans bénéfice proportionnel.

Q — Comment les trois interfaces communiquent-elles avec le backend ?

R — Chaque page HTML est un client statique qui appelle l'API Flask en HTTP/JSON via `fetch()` sur `http://localhost:5000` . `Flask-CORS` autorise ces appels cross-origin. Il n'y a pas de rendu serveur (pas de templates Jinja) : toute la mise à jour de l'UI se fait côté client en JavaScript à partir des réponses JSON.

Q — Le projet est-il "temps réel" au sens strict (WebSockets) ?

R — Non — c'est du **polling** : le frontend rappelle régulièrement certains endpoints (`refreshKPI` toutes les 15s, `fermer-retards` toutes les 30s, `buildEscalade` toutes les 2 min, rafraîchissement ticket actif toutes les 10s côté technicien). C'est plus simple à mettre en œuvre que des WebSockets et suffisant pour la fréquence de mise à jour souhaitée ; une évolution possible serait Server-Sent Events ou WebSockets pour du push serveur→client.

5.3 Base de données & modélisation

Q — Expliquez votre schéma en étoile : quelles sont les dimensions et les faits, et pourquoi ce découpage ?

R — Voir section 3 — dimensions = référentiels stables (secteur, équipement, technicien, temps), faits = événements (incident, ticket, historique). Ce découpage isole les données qui changent souvent (faits) de celles qui changent rarement (dimensions), et permet des agrégations simples par jointure dimension↔fait.

Q — Comment gérez-vous la cohérence entre `fact_incidents`, `fact_tickets` et `dim_technicien` lors d'un dispatch ?

R — L'assignation (`POST /api/dispatch/assigner`) effectue plusieurs `UPDATE` / `INSERT` dans la même requête de traitement : création du ticket dans `fact_tickets` , mise à jour de `fact_incidents.statut/technicien *` , et mise à jour de `dim_technicien.statut/ticket actif_id` . Ces opérations ne sont pas explicitement enveloppées dans une transaction SQL nommée dans le code actuel — `psycpg2` est en mode autocommit par défaut sauf configuration contraire ; un point d'amélioration serait d'envelopper ces écritures dans une transaction explicite (`BEGIN` / `COMMIT`) pour garantir l'atomicité en cas d'erreur partielle.

Q — Comment est calculé `duree_estimee_min` d'un incident, et pourquoi des plages différentes par type ?

R — À la génération (`generate_data.py`), chaque incident reçoit une durée tirée aléatoirement dans une plage propre à son `type_incident` (ex. `PANNE_ALIM` : 240-720 min, `ALARME_RESEAU` : 30-120 min), reflétant la criticité réelle de ce type de panne. Cette valeur sert ensuite de base au calcul des horaires de dispatch et au déclenchement de la clôture automatique.

5.4 Algorithme de dispatch

Q — Sur quels critères se base réellement le tri des techniciens proposés ?

R — Deux critères automatiques, dans cet ordre : (1) le technicien appartient-il au même secteur que l'incident, (2) la distance à vol d'oiseau (formule `111 * sqrt(Δlat² + Δlng²)` , ≈ km). La note de performance (`note_perf`) et les compétences sont affichées pour information mais n'entrent pas dans le tri automatique — c'est le superviseur qui valide le choix final.

Q — Pourquoi une distance "à vol d'oiseau" et pas une distance routière réelle (Google Maps, OSRM) ?

R — Calculer un itinéraire routier réel pour chaque paire technicien/incident impliquerait un appel à une API externe (payante ou à héberger) pour chacun des ~30 techniciens à chaque dispatch — coût et latence non justifiés pour un PFE de démonstration. La distance euclidienne convertie en km via `111 km/deg` est une approximation standard, correcte à l'échelle régionale, et suffisante pour classer les candidats par proximité relative.

Q — Comment est gérée une intervention qui dépasserait la journée de travail (après 18h) ?

R — Au moment de l'assignation, l'API calcule `heure_actuelle + duree_estimee_min` . Si ce total dépasse 18h00 (ou si la demande arrive hors horaires, avant 9h/après 18h), le ticket **n'est pas créé** : l'API répond `confirmation_requise: true` avec l'heure de fin estimée, et le frontend affiche une popup proposant de reporter l'intervention au **lendemain 09h00** (`POST /api/dispatch/assigner-j1`), insérée alors avec `date_assignment` = demain 09h00.

Q — Le système gère-t-il une pause déjeuner fixe dans le calcul des horaires ?

R — Non — ce point a été volontairement supprimé : seules les bornes 09h00-18h00 de la journée de travail sont prises en compte dans le calcul de `fin_eff` . La seule "pause" gérée dynamiquement est la pause terrain prise par le technicien pendant une intervention (voir 4.6), qui prolonge `duree_estimee_min` du ticket concerné mais ne modifie pas les horaires globaux de la journée.

5.5 Fonctionnalités temps réel (clôture auto, pause, Gantt)

Q — Comment fonctionne la clôture automatique des tickets en retard, et quelle est sa fréquence ?

R — Toutes les 30 secondes, `noc_admin.html` appelle `POST /api/tickets/fermer-retards` . L'API compare, pour chaque ticket actif (hors `COMPLETE` / `INCOMPLET` / `EN_PAUSE`), `date_assignment + duree_estimee_min` à l'heure actuelle. Si dépassé, le ticket passe `INCOMPLET` avec la cause "Durée estimée dépassée — clôture automatique NOC", `updated_by='SYSTEME'` , et le technicien redevient `DISPONIBLE` .

Q — Pourquoi exclure les tickets `EN_PAUSE` de la détection de retard ?

R — Parce qu'une pause terrain **prolonge déjà** `duree_estimee_min` (la durée de la pause y est ajoutée au moment du `END`). Si on ne les excluait pas, un ticket pourrait être fermé automatiquement **pendant** la pause elle-même (puisque `duree_pause_min` n'est ajoutée qu'à la fin de la pause), ce qui pénaliserait injustement le technicien.

Q — Que se passe-t-il concrètement avec un incident dont le ticket a été fermé automatiquement (`INCOMPLET`) ?

R — L'incident repasse `statut='INCOMPLET'` et reste visible sur la carte admin comme "bloqué". Le superviseur doit explicitement cliquer "Débloquer" (`POST /api/incidents<id>/debloquer`), qui le remet `OUVERT` pour qu'il puisse être redispaché normalement — aucun redispach automatique n'a lieu sans validation humaine.

Q — Une pause terrain peut-elle être prise plusieurs fois sur le même ticket ?

R — Non, c'est volontairement limité à **une seule pause par ticket** : l'API vérifie `duree_pause_min > 0` avant d'autoriser un nouveau `START` et renvoie une erreur 400 sinon. Ce choix évite les abus tout en restant réaliste (une pause de 15-30 min par intervention).

Q — Comment le Gantt affiche-t-il une journée passée vs la journée en cours ?

R — `GET /api/gantt?date=YYYY-MM-DD` accepte n'importe quelle date ; le frontend permet de naviguer jour par jour (boutons précédent/suivant + sélecteur de date). Pour le jour courant uniquement, le rafraîchissement automatique (30s) et la détection de retard sont actifs ; pour les jours passés, l'affichage est figé (lecture de l'historique).

5.6 KPI et calculs

Q — Comment est calculé le MTTR (Mean Time To Repair) affiché ?

R — Deux métriques liées sont calculées : `mttr_moyen_min` = `AVG(duree_resolution_min)` sur `fact_incidents` (durée entre détection et résolution, tous incidents résolus confondus), et `mttr_technique_min` = `AVG(duree_intervention_min)` sur `fact_tickets` où `statut_terrain = 'COMPLETE'` (durée d'intervention technique réelle). Le premier inclut le temps avant dispatch, le second mesure la performance terrain pure.

Q — D'où vient la "durée estimée moyenne par intervention" (~3h40) ?

R — C'est `ROUND(AVG(duree_estimee_min))` sur l'ensemble de `fact_incidents` (300 incidents), tous statuts confondus. Comme les incidents CRITIQUES (ex. PANNE_ALIM jusqu'à 720 min) tirent la moyenne vers le haut, le résultat avoisine 220 minutes ≈ 3h40.

Q — Le taux de résolution et les compteurs (critiques/majeurs/mineurs) sont-ils calculés en temps réel ?

R — Oui, à chaque appel de `GET /api/kpi` (toutes les 15 secondes côté dashboard), une requête SQL d'agrégation est exécutée sur l'état courant de la base — pas de cache ni de pré-calcul. Les "incidents critiques" affichés sont les CRITIQUE encore `OUVERT` ou `EN_COURS` (le compteur baisse dès qu'un ticket est créé pour eux ou qu'ils sont résolus).

5.7 Authentification & sécurité

Q — Comment gérez-vous l'authentification ?

R — De façon volontairement simplifiée : `login.html` propose deux profils (NOC Admin / Technicien Terrain), stockés dans `sessionStorage`. Côté technicien, un sélecteur permet ensuite de choisir lequel des 30 techniciens on incarne. Il n'y a pas de mot de passe ni de compte individuel — ce choix a été validé avec l'encadrant comme hors périmètre du PFE, une implémentation complète (table `dim_users`, hashage de mot de passe) étant prévue pour une phase ultérieure par une équipe de développement dédiée.

Q — L'API est-elle sécurisée (authentification, autorisation) ?

R — Non — `app.py` ne fait aucune vérification d'identité sur les endpoints ; n'importe quel client connaissant l'URL peut appeler l'API. C'est cohérent avec le périmètre "démonstrateur local" du projet (`DB_CONFIG` en clair, CORS ouvert). Pour une mise en production, il faudrait au minimum une authentification par token (JWT/session) et une vérification de rôle (admin vs technicien) sur chaque endpoint sensible.

Q — Les mots de passe/identifiants de la base de données sont-ils protégés ?

R — Actuellement en clair dans `DB_CONFIG` (`app.py`) ou en variables d'environnement avec valeurs par défaut (`etl_pipeline.py`). Pour la production, ces secrets devraient être externalisés (variables d'environnement obligatoires, fichier `.env` non commité, ou gestionnaire de secrets).

5.8 Limites, robustesse et perspectives d'évolution

Q — Quelles sont les limites actuelles du projet, en toute honnêteté ?

R — (1) données simulées, pas connectées à un système de supervision réel (U2000) ; (2) pas d'authentification individuelle ; (3) distance euclidienne au lieu d'un itinéraire routier réel ; (4) polling plutôt que push WebSocket ; (5) configuration DB en dur ; (6) pas de transaction explicite sur les écritures multi-tables du dispatch ; (7) pas de tests automatisés.

Q — Si vous aviez plus de temps, quelles seraient les prochaines étapes ?

R — Authentification complète par technicien (la table `dim_users` est déjà prévue dans le schéma historique) ; intégration à un flux d'incidents réel (webhook/API du système de supervision) ; distance routière réelle via une API de cartographie ; passage à des WebSockets/SSE pour le temps réel ; notifications push mobiles pour les techniciens ; tableau de prévision de charge par secteur (apprentissage automatique sur l'historique des incidents) ; tests unitaires/d'intégration sur l'API.

Q — Le système est-il scalable à plus de techniciens/secteurs/incidents ?

R — Le modèle de données et les requêtes SQL ne sont pas spécifiques aux volumes actuels (10 secteurs / 120 équipements / 30 techniciens / 300 incidents) — augmenter ces nombres ne change pas la structure. La limite pratique viendrait du polling fréquent (15-30s) si le nombre de clients connectés augmentait fortement, et de l'absence d'index explicites au-delà des clés primaires/étrangères par défaut de PostgreSQL — à auditer avec `EXPLAIN ANALYZE` si le volume de données croît significativement.

Q — Pourquoi avoir simulé les données plutôt que d'utiliser un jeu de données public ?

R — Le projet modélise spécifiquement le contexte Huawei Maroc (secteurs géographiques réels du pays, types de pannes télécom, techniciens avec compétences/secteurs/horaires) — aucun jeu de données public ne correspond à ce schéma précis. La génération contrôlée (`generate_data.py`) permet aussi de garantir un jeu de données cohérent et reproductible pour la démonstration (mêmes secteurs/équipements à chaque régénération, incidents aléatoires mais dans des plages réalistes par type).